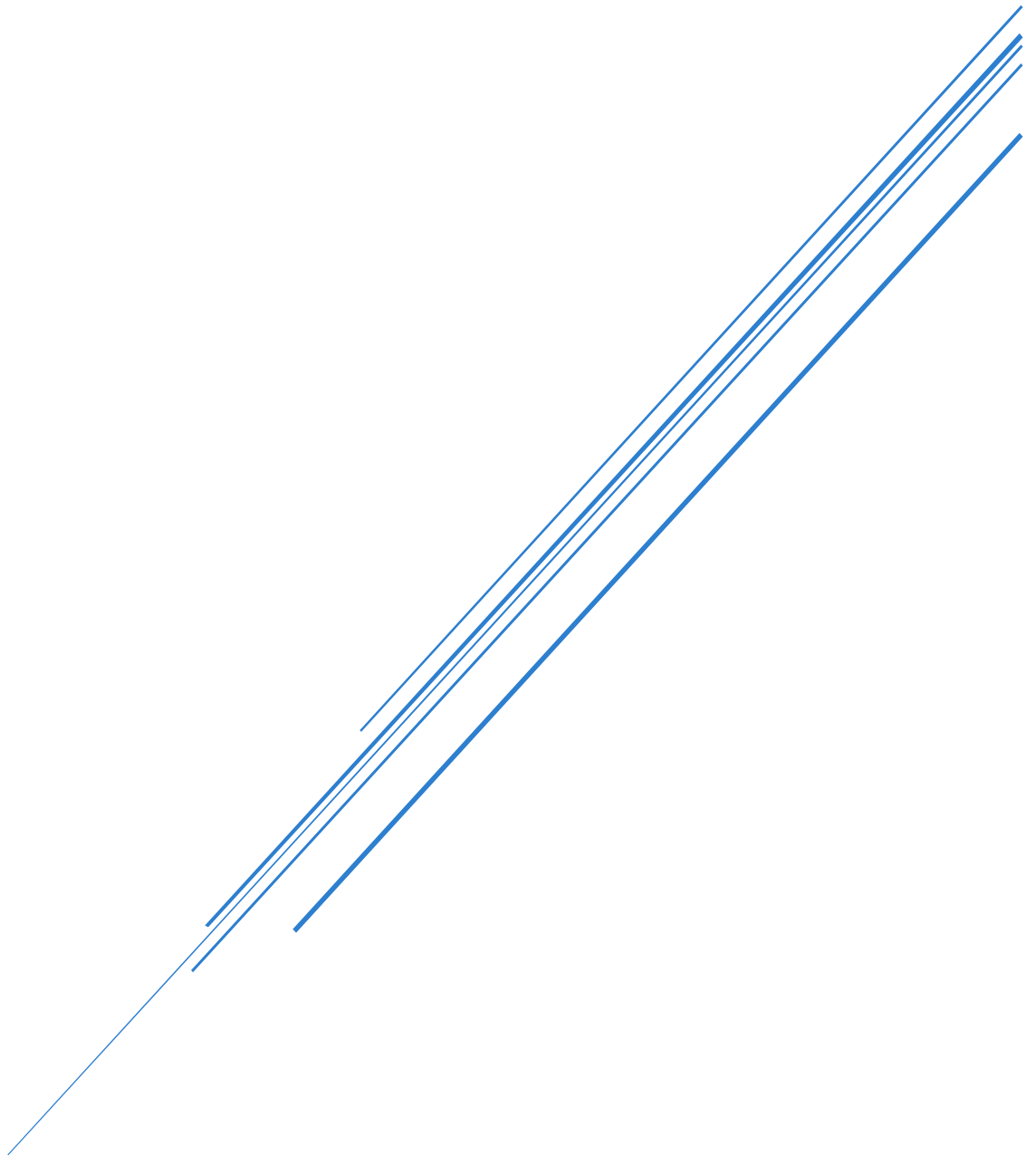


XISD5319

Task 2

Github: [mzamon/Shuttle_Management_System](https://github.com/mzamon/Shuttle_Management_System)



Rosebank
DISD

Table f Contents

Github: mzamon/Shuttle_Management_System.....	0
TASK 2: REQUIREMENT ANALYSIS DOCUMENT	4
NBK Travel Shuttle Booking Management System	4
Introduction:.....	4
1.1 Organisational Background	4
1.2 Problem Statement	4
1.3 Project Objectives	5
1.4 Scope & Constraints.....	6
1.5 Stakeholders	6
2. Solution Domain: Functional Requirements & UML Use Case Diagrams	7
2.1 Functional Requirements Table	7
2.2 UML Use Case Diagram.....	9
2.2.1 Actors.....	9
3. Logical System Model	10
3.1 Method Naming Conventions.....	12
4. Class Diagrams	13
4.1 Class Diagram Table	13
TASK 3: SYSTEM DESIGN DOCUMENT.....	18
Introduction	18
System Purpose	18
Technical Overview:.....	18
Business Goals:	18
5. Low-Level Design Diagram.....	19
6. Input Interactions (GUI Input Menus & Forms)	20
6.1 Input Menu Hierarchy	20
6.2 Input Form Specifications	22
6.2.1 New Booking Form	22
6.2.2 New / Edit Customer Form	22
6.2.3 Assign Driver & Vehicle Form	23
7. Request Interactions (GUI Output / Reports).....	24
7.1 Request Interaction Summary.....	24

7.2 Report Parameter Forms.....	25
7.2.1 Trip Report Parameters	25
7.2.2 Revenue Report Parameters	26
8. Database Design	27
9. Database Tables.....	28
Table Name: bookings	28
Table Name: customers	28
Table Name: drivers	29
Table Name: vehicles	29
Table Name: schedules	29
Table Name: invoices	30
Table Name: notifications.....	30
10. ERD Diagrams	31
ERD Diagram 1: Core Transaction Entities	31
ERD Diagram 2 - Supporting Entities.....	32
11. System Reports Design	33
11.1 Report Design Notes.....	34
12. Prototype	35
12.1 Dashboard.....	35
12.2 Bookings.....	35
12.3 Schedule	36
12.4 Customers.....	37
12.5 Reports	37
12.6 Invoices	38
13. Appendix.....	39
Appendix A: UML Use Case	40
Appendix B: Class Diagram	41
Appendix C: Low-Level Design Diagram	42
References.....	43

Team Members:

Full Name	Student Number	Role
Shenice Wood	ST10447209	Project Manager
Murendeni Makhavhu	ST10377430	Database Administrator
Thandiwe Sibeko	ST10446961	System Analyst
Matome Maopye	ST10341694	UI/UX Designer& QA Tester
Mzamo Richmond Ndlovu	ST10455453	Software Developer

TASK 2: REQUIREMENT ANALYSIS DOCUMENT

NBK Travel Shuttle Booking Management System

Introduction:

This section presents a complete study of the problem domain identified within NBK Travel (nbktravel.co.za), a shuttle and transport service company operating across South Africa. The analysis expands upon the introduction provided in the Task 1 Project Plan and delivers a full specification of the operational challenges, system requirements, and project constraints that the Shuttle Booking Management System is designed to address (IIE, 2026).

1.1 Organisational Background

NBK Travel is a South African shuttle and transport services company providing airport transfers, corporate travel and general commuter transport. The company operates a fleet of vehicles across multiple routes and employs a team of professional drivers to serve both individual and corporate clients. NBK Travel's brand identity is built on reliability, punctuality and personalised service values that its current manual booking processes directly undermine. Prior to this development initiative, all booking operations were conducted through telephone calls, paper based registries and personal spreadsheets. This approach creates critical inefficiencies and competitive disadvantages in an increasingly digital marketplace. As client volumes grow, the manual system becomes progressively less sustainable, threatening both operational integrity and NBK Travel's reputation in the South African transport sector. The Shuttle Booking Management System has been commissioned to digitise and centralise NBK Travel's operational workflows through a web based platform built using HTML, CSS, and JavaScript on the frontend, with a PHP backend and a MySQL relational database managed via phpMyAdmin on a WAMP Server environment (IIE, 2026).

1.2 Problem Statement

The following specific operational deficiencies were documented during stakeholder engagement and requirements elicitation with NBK Travel management:

- Double-bookings and scheduling conflicts: Without a real-time centralised registry, NBK Travel cannot detect simultaneous reservations for the same driver or vehicle, resulting in service failures and reputational damage.
- Fragmented customer records: Customer histories are dispersed across paper files and personal spreadsheets, making repeat-client recognition, preference retrieval, and proactive communication impractical.
- Absence of business intelligence: Management cannot generate accurate reports on trip volumes, revenue trends, driver utilisation, or route profitability, severely constraining both operational and strategic decision-making.
- No digital invoicing: Customer-facing invoices are produced manually, reducing professionalism and complicating dispute resolution.
- No notification system: Booking confirmations and driver reminders are communicated informally, leading to missed trips.
- No driver visibility dashboard: Drivers have no centralised view of their assigned trips, leading to schedule confusion and missed assignments.

1.3 Project Objectives

- Booking Management Module: A digital system to create, store, modify, and retrieve all trip bookings with real-time availability verification.
- Scheduling System: Automated assignment of drivers and vehicles to bookings, with built-in conflict detection and resolution.
- Customer Database: Comprehensive records for new, repeat, and potential clients including contact details, booking history, and preferences.
- Reporting Module: On-demand reports covering trips per day/week, revenue summaries, driver utilisation, and route performance.
- Invoice Generator: Automated production of client invoices in PDF format using html2pdf.js.
- Notification Simulation: System-triggered booking confirmations and reminders via simulated email/SMS channels.
- Driver Dashboard: A dedicated view for drivers to see assigned trips, departure times, and pickup/drop-off details.

1.4 Scope & Constraints

- The system is a web-based application built with HTML/CSS/JavaScript (front end) and PHP (back end) hosted via WAMP SERVER locally.
- The database is implemented in MySQL, managed via phpMyAdmin.
- Version 1.0 excludes GPS tracking, live payment gateway integration, and multi-branch fleet management.
- All development tools are open-source or institutionally provided by Rosebank College — no commercial licensing costs are incurred.
- The project is executed within the XISD5319 WIL academic framework with a delivery timeline.
- GitHub repository: https://github.com/mzamon/Shuttle_Management_System/

1.5 Stakeholders

Stakeholder	Role in System	Primary Interest
NBK Travel Management	Commission, approve and use reports	Operational oversight, revenue tracking
NBK Travel Administrator	Primary system user bookings, scheduling, reports	Efficiency, accuracy, reduced manual workload
NBK Travel Drivers	View assigned trips, mark trips complete	Clear schedule visibility
NBK Travel Customers	Submit requests, receive invoices and confirmations	Clear schedule visibility
Development Team	Build, test and deliver the system	Academic credit, technical skill development
Rosebank College (IIE)	Academic assessor	WIL module learning outcomes

2. Solution Domain: Functional Requirements & UML Use Case Diagrams

This section provides the logical description of all functional requirements of the proposed NBK Travel Shuttle Booking Management System. Requirements are presented in the standard three-column format showing the active participant, the system function performed, and the passive participant receiving the output.

2.1 Functional Requirements Table

The following table documents all 20 functional requirements mapped across six sub-systems. The system is implemented using PHP (backend), MySQL (database), and HTML/CSS/JavaScript (frontend).

Participant(Active Actor)	Sub-System	Function of the System	Participant (Passive Actor)
Administrator	Booking Management	Create a new booking capturing customer name, pickup/drop-off location, date, time, and number of passengers	Customer
Administrator	Booking Management	View all bookings in a table with filter by date and status	—
Administrator	Booking Management	Edit an existing booking to update details or reschedule	Customer
Administrator	Booking Management	Cancel/delete a booking and log the cancellation reason	Customer
Customer	Booking Management	Submit a shuttle booking request via the web form	Administrator
Administrator	Scheduling System	Assign a driver and vehicle to a confirmed booking	Driver
System	Scheduling System	Detect and flag overlapping driver/vehicle assignments automatically	Administrator

Participant(Active Actor)	Sub-System	Function of the System	Participant (Passive Actor)
Administrator	Scheduling System	View daily and weekly schedule grid for all drivers	Driver
Driver	Scheduling System	View assigned trips via the driver dashboard	Administrator
Driver	Scheduling System	Mark a trip as completed upon passenger delivery	Administrator
Administrator	Customer Records	Add or update a repeat customer profile (name, contact, preferences)	Customer
Administrator	Customer Records	Search for a customer by name/phone and view full booking history	Customer
System	Customer Records	Automatically match a new booking to an existing customer record	Customer
Administrator	Reporting Module	Generate a trip count report by day or week	—
Administrator	Reporting Module	Generate a revenue summary report for a selected period	—
Administrator	Reporting Module	View most frequent customers ranked by booking count	Customer
Administrator	Invoice Generator	Generate a PDF invoice/receipt for a completed booking using html2pdf.js	Customer
System	Invoice Generator	Auto-populate invoice with booking details, fare, and NBK Travel company info	Customer

Participant(Active Actor)	Sub-System	Function of the System	Participant (Passive Actor)
System	Notification Simulation	Log a simulated SMS/email confirmation when a booking is created or updated	Customer
System	Notification Simulation	Log a simulated SMS/email reminder to the driver before trip departure	Driver

2.2 UML Use Case Diagram

The UML Use Case Diagram below depicts the NBK Travel Shuttle Booking Management System boundary, all identified use cases (UC01–UC13), and three actors (Administrator, Driver, Customer) together with one automated actor (System).

2.2.1 Actors

Administrator (Active): NBK Travel's primary system user. Responsible for all booking creation, scheduling, customer management, report generation, and invoice production.

Driver (Active): NBK Travel shuttle driver. Views assigned trips via the driver dashboard and marks trips complete.

Customer (Passive): NBK Travel client. Submits booking requests and receives invoices and notification confirmations.

System (Active -- Automated): Automated processes including conflict detection, customer record matching, and notification simulation.

Refer to Appendix A: UML Use Case Diagram (Screenshot)

3. Logical System Model

The Logical System Model defines the precise mapping between user interface actions (GUI inputs and outputs), the PHP system methods invoked in response, and the underlying MySQL database tables affected. This model is the operational heart of the system specification, every row traces a complete interaction path from user action through to the data layer (IIE, 2026).

All system methods are implemented as PHP functions exposed through REST-aligned endpoints. Database operations are executed via MySQL prepared statements against the MySQL schema managed in phpMyAdmin.

GUI Input	GUI Output	System Method	Process (PHP)	Entity Relationship (MySQL Table)
Enter booking details (customer, locations, date/time, pax)	Booking confirmation displayed	createBooking(\$customerId, \$pickup, \$dropoff, \$datetime, \$pax)		bookings → customers (FK)
Select booking from list	Booking detail view rendered	getBookingById(\$bookingId)		bookings table
Update booking fields & submit	Updated record confirmation	updateBooking(\$bookingId, \$fields)		bookings table
Click cancel, enter reason	Cancellation status logged	cancelBooking(\$bookingId, \$reason)		bookings table
Select driver & vehicle for booking	Schedule updated / conflict modal shown	assignDriverVehicle(\$bookingId, \$driverId, \$vehicleId)		schedules → drivers (FK) → vehicles (FK)

GUI Input	GUI Output	System Process (PHP METHOD)	Entity Relationship (MySQL Table)
No input	Conflict warning highlighted in red	<code>detectConflict(\$driverId, \$vehicleId, \$datetime, \$duration)</code>	schedules table
Select date/week for schedule view	Calendar/grid view rendered	<code>getSchedule(\$startDate, \$endDate)</code>	schedules → bookings
Driver clicks 'Mark Complete'	Trip status updated to 'completed'	<code>completeTrip(\$bookingId, \$driverId)</code>	bookings → schedules
Enter customer details, submit	Customer profile saved	<code>createOrUpdateCustomer(\$name, \$phone, \$email)</code>	customers table
Type name or phone in search box	Customer profile + booking history list	<code>searchCustomer(\$query)</code>	customers → bookings
No input (triggered on booking create)	Booking linked to existing customer	<code>matchCustomerRecord(\$name, \$phone)</code>	customers → bookings
Select report period (date range)	Trip count bar chart + table	<code>generateTripReport(\$start, \$end)</code>	bookings table

GUI Input	GUI Output	System Process (PHP METHOD)	Entity Relationship (MySQL Table)
Select revenue period	Revenue summary with totals	generateRevenueReport(\$start, \$end)	bookings → fares
No input	Top customers ranked table	getTopCustomers(\$limit)	customers → bookings
Select completed booking	PDF invoice downloaded via html2pdf.js	generateInvoice(\$bookingId)	bookings → customers → fares
No input (triggered on events)	Notification log entry written	logNotification(\$type, \$recipientId, \$msg, \$channel)	notifications table

3.1 Method Naming Conventions

All PHP system methods follow a camelCase verb-entity convention: [verb][Entity](\$parameters). Methods map directly to SQL operations (SELECT, INSERT, UPDATE, DELETE) executed by prepared statements. This convention was agreed during the System Design phase (T3) to ensure consistency across all team member's contributions.

4. Class Diagrams

Class diagrams model the static structure of the NBK Travel Shuttle Booking Management System. Each entity class maps to a MySQL database table implemented via phpMyAdmin. Properties include attributes with their MySQL data types, methods represent the PHP functions available on each class.

4.1 Class Diagram Table

Entity Class (UML)	Properties (MySQL) & Methods (PHP)	Related To:
Booking	- bookingId : INT (PK, AI) - customerId : INT (FK → customers) - driverId : INT (FK → drivers) - vehicleId : INT (FK → vehicles) - pickupLocation : VARCHAR(100) - dropoffLocation : VARCHAR(100) - bookingDate : DATETIME - passengers : INT - status : ENUM('pending','confirmed','completed','cancelled') - cancellationReason : VARCHAR(255) - fareAmount : DECIMAL(8,2) - createdAt : DATETIME - PHP Methods - - createBooking() - updateBooking() - cancelBooking() - completeTrip() - getBookingById()	- Customer (Many-to-One) - Driver (Many-to-One) - Vehicle (Many-to-One) - Invoice (One-to-One) - Notification (One-to-Many)
Customer	- customerId : INT (PK, AI) - fullName : VARCHAR(100) - phoneNumber : VARCHAR(20) - emailAddress : VARCHAR(100) - preferences : VARCHAR(255) - createdAt : DATETIME	- Booking (One-to-Many) - Notification (One-to-Many)

Entity Class) (UML	Properties (MySQL) & Methods (PHP)	Related To:
	- PHP Methods - - createCustomer() - updateCustomer() - searchCustomer() - getBookingHistory() - getTopCustomers()	Invoice (One-to-Many)
Driver	- driverId : INT (PK, AI) - fullName : VARCHAR(100) - licenceNumber : VARCHAR(30) - phoneNumber : VARCHAR(20) - status : ENUM('available','on-trip','off-duty') - PHP Methods - - assignToBooking() - markTripComplete() - getAssignedTrips() - getSchedule()	- Booking (One-to-Many) - Schedule (One-to-Many)
Vehicle	- vehicleId : INT (PK, AI) - registrationNumber : VARCHAR(20) - make : VARCHAR(50) - model : VARCHAR(50) - capacity : INT - status : ENUM('available','in-use','maintenance') - PHP Methods - - assignToBooking() - checkAvailability() - getVehicleSchedule()	- Booking (One-to-Many) - Schedule (One-to-Many)

Entity Class) (UML	Properties (MySQL) & Methods (PHP)	Related To:
Schedule	- scheduleId : INT (PK, AI) - bookingId : INT (FK → bookings) - driverId : INT (FK → drivers) - vehicleId : INT (FK → vehicles) - scheduledStart : DATETIME - scheduledEnd : DATETIME - conflictFlag : TINYINT(1) - PHP Methods - - createEntry() - detectConflict() - getByDateRange() - updateStatus()	- Booking (One-to-One) - Driver (Many-to-One) - Vehicle (Many-to-One)
Invoice	- invoiceId : INT (PK, AI) - bookingId : INT (FK → bookings) - customerId : INT (FK → customers) - invoiceDate : DATETIME - subtotal : DECIMAL(8,2) - taxAmount : DECIMAL(8,2) - totalAmount : DECIMAL(8,2) - pdfPath : VARCHAR(255) - PHP Methods - - generateInvoice() - exportToPDF() - html2pdf.js - getInvoiceByBooking()	- Booking (One-to-One) - Customer (Many-to-One)
Notification	- notificationId : INT (PK, AI) - recipientType : ENUM('customer','driver') - recipientId : INT	Customer (Many-to-One)

Entity Class (UML)	Properties (MySQL) & Methods (PHP)	Related To:
	- channel : ENUM('sms','email') - messageBody : VARCHAR(500) - sentAt : DATETIME - status : ENUM('logged','failed') - PHP Methods - - logNotification() - getByRecipient()	- Driver (Many-to-One) - Booking (Many-to-One)

Refer to Appendix B: Class Diagram

TASK 3: SYSTEM DESIGN DOCUMENT

Introduction

This document presents the complete System Design for the NBK Travel Shuttle Booking Management System a web based platform commissioned to replace NBK Travel's manual, paper based booking operations with a centralised, digital management solution. The design continues directly from the Requirement Analysis document and translates all functional requirements, use cases and class relationships into a concrete, implementable technical specification (IIE, 2026).

System Purpose

The Shuttle Booking Management System enables NBK Travel's administrator to create and manage shuttle bookings, assign drivers and vehicles with automated conflict detection, maintain a structured customer database and generate operational reports all within a single, browser accessible web interface. Drivers access a dedicated dashboard to view assigned trips and mark trips as completed.\

Technical Overview:

The system is implemented as a three tier web application:

Presentation Layer: Plain HTML5, CSS3 and JavaScript (ES6) ensuring browser compatibility and zero licensing cost.

Business Logic Layer: PHP 8.x backend with MySQLi prepared statements for all database interactions.

Data Layer: MySQL Community Edition managed via phpMyAdmin on a WAMP Server local environment.

Business Goals:

Eliminate double bookings through real-time automated conflict detection.

Centralise all NBK Travel operational data in a single, structured database.

Enable management to access business intelligence reports on demand.

Automate invoice generation and notification logging to reduce administrative workload.

Expected Impact:

Post-deployment, NBK Travel is expected to eliminate scheduling conflicts, reduce manual administrative effort by an estimated 15–20 hours per week and achieve full cost recovery within 16–21 weeks of deployment based on labour cost savings alone (IIE, 2026).

5. Low-Level Design Diagram

- The low-level design diagram models the internal structure of the NBK Travel Shuttle Booking Management System. It consists of three components, as specified in the module requirements (IIE, 2026):
- Actors: The same actors identified in the Task 2 use case diagrams, Administrator, Driver, Customer, and System (automated).
- Functions: System functionality is modelled using use cases. Arrows indicate (include) and (extend) relationships between use cases. Each use case is briefly described.
- Database: The MySQL tables referenced in each use case are indicated using a circle/oval annotation. Table names and their meaning are labelled.

Refer to Appendix C: Low-Level Design Diagram

6. Input Interactions (GUI Input Menus & Forms)

Input interactions represent all data controls used to read data into the NBK Travel system. This is the first complete reference to the GUI and specifies all input interactions in detail. The structured annotated menu list below indicates the hierarchical relationship between input menus, as specified in Option 1 of the module requirements (IIE, 2026).

6.1 Input Menu Hierarchy

Menu Level	Menu Item #1	Menu Item #2	Menu Item #3	Menu Item #4
1st Level	Booking Management			
2nd Level	New Booking Form	Manage Bookings		
3rd Level	Customer Name	Pickup Location	Drop-off Location	Date & Time
3rd Level	No. of Passengers	Fare Amount	Status	
1st Level	Schedule Management			
2nd Level	Assign Driver & Vehicle	View Schedule		
3rd Level	Select Booking	Select Driver	Select Vehicle	Scheduled Date/Time
1st Level	Customer Records			
2nd Level	New Customer Form	Search Customer	Edit Customer	
3rd Level	Full Name	Phone Number	Email Address	Preferences

Menu Level	Menu Item #1	Menu Item #2	Menu Item #3	Menu Item #4
1st Level	Reports			
2nd Level	Trip Report	Revenue Report	Top Customers	Driver Utilisation
3rd Level	Date Range (Start)	Date Range (End)	Group By (Day/Week)	
1st Level	Invoice Generator			
2nd Level	Select Completed Booking	Generate PDF (html2pdf.js)		
1st Level	Driver Dashboard			
2nd Level	View Assigned Trips	Mark Complete Trip		
1st Level	Authentication			
2nd Level	Login Form			
3rd Level	Username	Password		

6.2 Input Form Specifications

6.2.1 New Booking Form

Field Name	Data Type (PHP/MySQL)	Validation	Implementation Notes
Customer Name	VARCHAR(100)	Required, min 2 chars	Auto-suggest from customers table via PHP AJAX
Pickup Location	VARCHAR(100)	Required	Free-text or predefined NBK Travel route dropdown
Drop-off Location	VARCHAR(100)	Required	Free-text address field
Booking Date	DATE	Required, not in past	HTML5 date picker; PHP server-side re-validation
Booking Time	TIME	Required	HTML5 time picker
No. of Passengers	INT	Required, 1–50	HTML number input with min/max attributes
Fare Amount	DECIMAL(8,2)	Required, > 0	Auto-calculated or manually entered by admin
Status	ENUM	Default: 'pending'	PHP SELECT dropdown: pending/confirmed/cancelled/completed

6.2.2 New / Edit Customer Form

Field Name	Data Type (PHP/MySQL)	Validation	Notes
Full Name	VARCHAR(100)	Required, min 2 chars	

Field Name	Data Type (PHP/MySQL)	Validation	Notes
Phone Number	VARCHAR(20)	Required, 10 digits	Format: 0XXXXXXXXX; PHP regex validation
Email Address	VARCHAR(100)	Optional, valid email	Used for simulated email notification
Preferences	VARCHAR(255)	Optional	E.g. preferred driver, vehicle type, NBK Travel account notes

6.2.3 Assign Driver & Vehicle Form

Field Name	Data Type	Validation	Notes
Booking Reference	INT (FK)	Required, must exist	Auto-populated from booking selection
Driver	INT (FK)	Required	PHP SELECT of drivers with status='available'
Vehicle	INT (FK)	Required	PHP SELECT of vehicles with status='available'
Scheduled Start	DATETIME	Required	Auto-populated from booking date/time
Scheduled End	DATETIME	Required	Calculated from estimated trip duration
Conflict Flag	TINYINT(1)	System-set	PHP detectConflict() sets TRUE if overlap found; form rejected

7. Request Interactions (GUI Output / Reports)

Request interactions represent all service requests placed on the NBK Travel system that produce functional outputs, including screen displays, printed reports, and PDF downloads. These interactions refer to output menus and parameter forms used to specify the services provided by the system (IIE, 2026).

7.1 Request Interaction Summary

Request Output /	Triggered By	Output Format	Data Source (MySQL)
Booking List View	Admin selects 'View Bookings'	Screen — HTML table with pagination	bookings JOIN customers
Booking Detail View	Admin clicks a booking row	Screen — PHP detail card	bookings + customers + drivers + vehicles
Schedule Grid (Daily)	Admin selects a date	Screen — HTML/CSS calendar grid	schedules JOIN bookings JOIN drivers
Schedule Grid (Weekly)	Admin selects a week	Screen — weekly HTML grid	schedules JOIN bookings JOIN drivers
Conflict Alert	PHP detectConflict() on assignment	Screen — red modal/alert in JavaScript	schedules table
Customer Profile & History	Admin searches by name/phone	Screen — profile + booking history list	customers JOIN bookings
Trip Count Report	Admin selects date range + group by	Screen chart + (Chart.js) print view	bookings table (bookingDate, status)

Request Output /	Triggered By	Output Format	Data Source (MySQL)
Revenue Summary Report	Admin selects period	Screen (Chart.js) chart + print view	bookings.fareAmount grouped by period
Top Customers Report	Admin selects limit	Screen — HTML ranked table	customers JOIN bookings COUNT(*)
Invoice / Receipt (PDF)	Admin clicks 'Generate Invoice'	PDF download via html2pdf.js	bookings + customers + invoices
Notification Log	System on booking events	Screen — HTML log table (read-only)	notifications table
Driver Trip List	Driver logs in to dashboard	Screen — driver's assigned trips	schedules JOIN bookings WHERE driverId

7.2 Report Parameter Forms

7.2.1 Trip Report Parameters

Field	Type	Validation	Purpose
Start Date	DATE	Required	Filter period start
End Date	DATE	Required, start >=	Filter period end
Group By	ENUM (Day/Week/Month)	Required	Chart granularity
Output	ENUM (Screen/Print)	Default: Screen	Rendering target

7.2.2 Revenue Report Parameters

Field	Type	Validation	Purpose
Start Date	DATE	Required	Filter period start
End Date	DATE	Required, >= start	Filter period end
Currency	VARCHAR (10)	Default: ZAR	Display formatting
Output	ENUM (Screen/Print/PDF)	Default: Screen	Rendering target via html2pdf.js

8. Database Design

The database design is derived directly from the Class Diagram (Section 4) and the Logical System Model (Section 3). Each entity class becomes a MySQL table, implemented via phpMyAdmin as part of the Database Development task (T5). All tables are normalised to at least Third Normal Form (3NF) to eliminate redundancy and ensure data integrity (Connolly and Begg, 2015).

DBMS: MySQL (Community Edition) — managed via phpMyAdmin / XAMPP

Backend: PHP with MySQLi prepared statements

Frontend: Plain HTML / CSS / JavaScript

Normalisation: Minimum 3NF - transitive dependencies removed

Key types: Primary Keys (PK), Foreign Keys (FK), Unique constraints, Check constraints

GitHub: https://github.com/mzamon/Shuttle_Management_System

9. Database Tables

Each table below presents the primary key, foreign key references, and data fields. Three fictitious but realistic sample rows are included per table to validate the design. All tables are implemented as MySQL scripts stored in database.

Table Name: bookings

bookingId (PK)	customerId (FK)	driverId (FK)	vehicleId (FK)	pickupLocation	dropoffLocation	bookingTime
1	3	2	1	14 Main Rd, JHB	Bela-Bela	2026-06-07:30
2	1	1	3	Sandton City	Mossel Bay	2026-06-09:00
3	5	3	2	Midrand	Bloemfontein	2026-06-06:45

Table Name: customers

customerId (PK)	fullName	phoneNumber	emailAddress	preferences
1	Shenice Wood	0608178335	jessicawood07@icloud.com	Window seat
2	Zoe Mokoena	0712345678	ZM@nbktravel.co.za	Early morning pickup preferred
3	Matt Maopye	0834567890	matt.n@work.co.za	Requires receipt always

Table Name: drivers

driverId (PK)	fullName	licenceNumber	phoneNumber	status
1	Joyce Dlamini	PDP-JHB-12345	0761234567	available
2	joe Khumalo	PDP-JHB-67890	0829876543	on-trip
3	Maria Zamani	PDP-JHB-24680	0734455667	available

Table Name: vehicles

vehicleId (PK)	registration	make	model	capacity	status
1	RY 12 RF GP	Toyota	Quantum	14	available
2	RXX 719 GP	Mercedes-Benz	Vito	8	in-use
3	ZWC 355 NW	Hyundai	H1	7	available

Table Name: schedules

scheduleId (PK)	bookingId (FK)	driverId (FK)	vehicleId (FK)	scheduledStart	scheduledEnd	conflictFlag
1	1	2	1	2026-06-01 07:30	2026-06-01 09:00	0
2	2	1	3	2026-06-02 09:00	2026-06-02 09:45	0
3	3	3	2	2026-06-03 06:45	2026-06-03 08:30	0

Table Name: invoices

invoiceId (PK)	bookingId (FK)	customerId (FK)	invoiceDate	subtotal	taxAmt	totalAmt	pdfPath
1	2	1	2026-06-02 10:05	R156.52	R23.48	R180.00	/invoices/INV001.pdf
2	5	3	2026-06-05 14:30	R391.30	R58.70	R450.00	/invoices/INV002.pdf
3	7	2	2026-06-07 09:15	R538.26	R81.74	R620.00	/invoices/INV003.pdf

Table Name: notifications

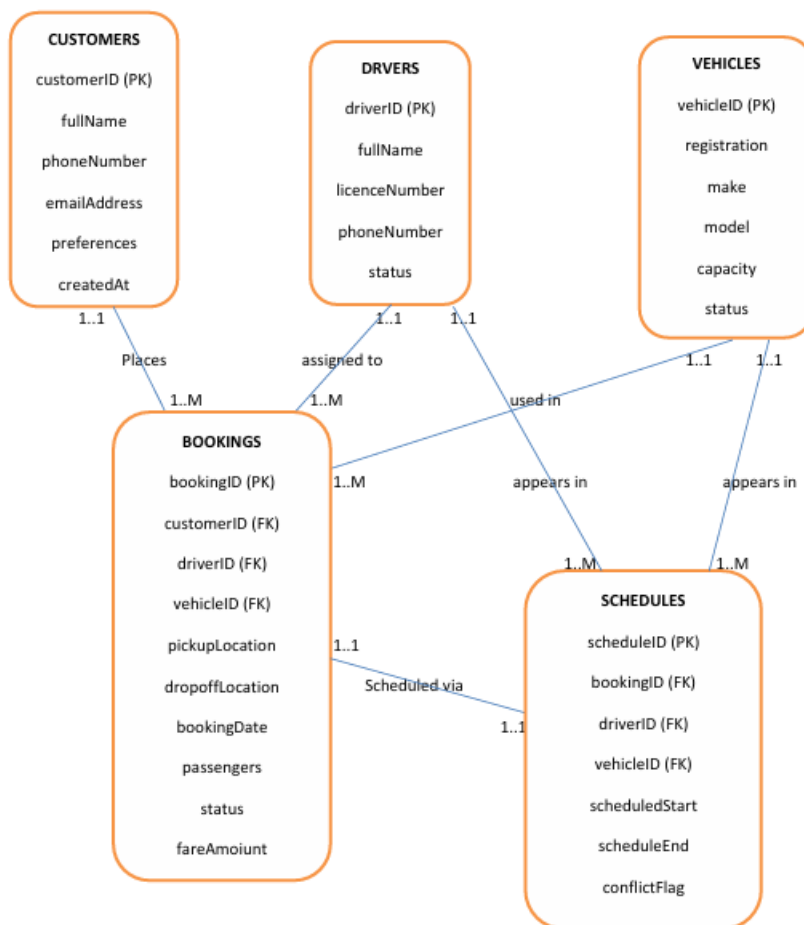
notifId (PK)	recipType	recipId	channel	messageBody	sentAt	status
1	customer	1	email	Booking #1 confirmed for 2026-06-01 07:30	2026-05-28 14:22	logged
2	driver	2	email	Trip #1 assigned. Pickup 14 Main Rd at 07:30	2026-05-28 14:22	logged
3	customer	3	email	Invoice INV001 generated for your trip.	2026-06-02 10:06	logged

10. ERD Diagrams

The Entity Relationship Diagrams (ERDs) below model the structural relationships between all MySQL tables in the NBK Travel Shuttle Booking Management System. Standard crow's-foot notation is used: rectangles for tables, lines for relationships, crow's-foot symbols for cardinality (one-to-many, one-to-one). Diagrams were produced using draw.io and are stored in the /docs/diagrams/ folder of the GitHub repository.

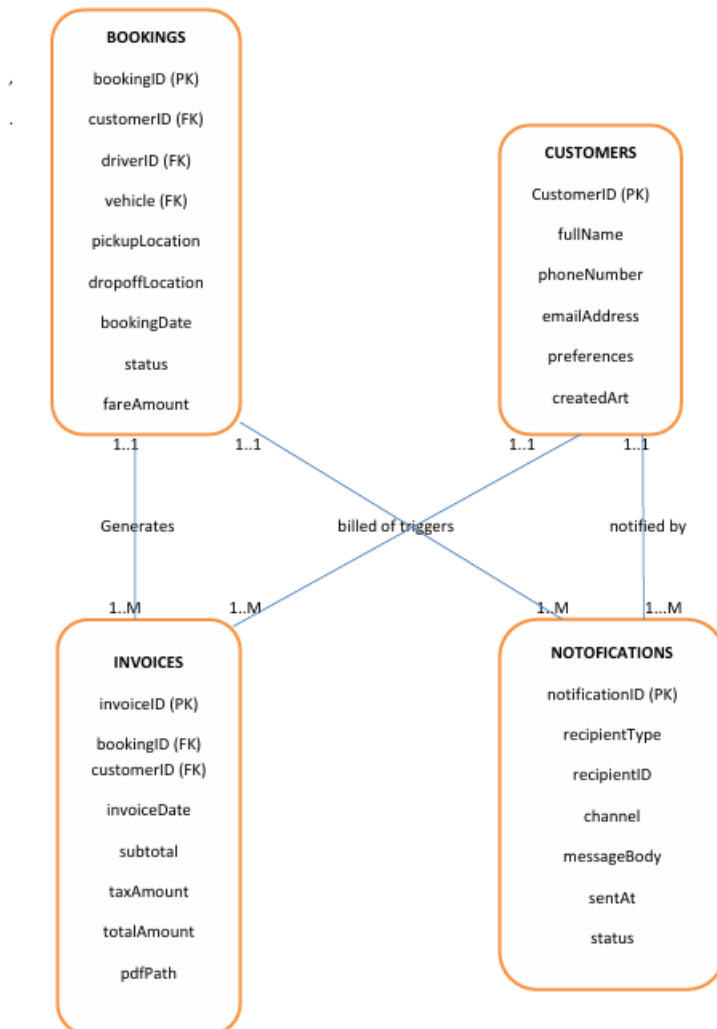
ERD Diagram 1: Core Transaction Entities

This ERD models the primary transactional relationships between the bookings, customers, drivers, vehicles, and schedules tables. These five tables form the operational core of the NBK Travel system.



ERD Diagram 2 - Supporting Entities

This ERD models the relationships between the bookings, invoices, and notifications tables, capturing the post-trip financial and communication record keeping structure.



11. System Reports Design

The following reports have been identified as required system outputs for NBK Travel. Each report specifies the data source (MySQL tables), the presentation format, the intended audience, and the PHP method responsible for generating the output. All report queries are optimised with indexes on bookingDate, customerId, and driverId for acceptable performance as NBK Travel's booking volume grows.

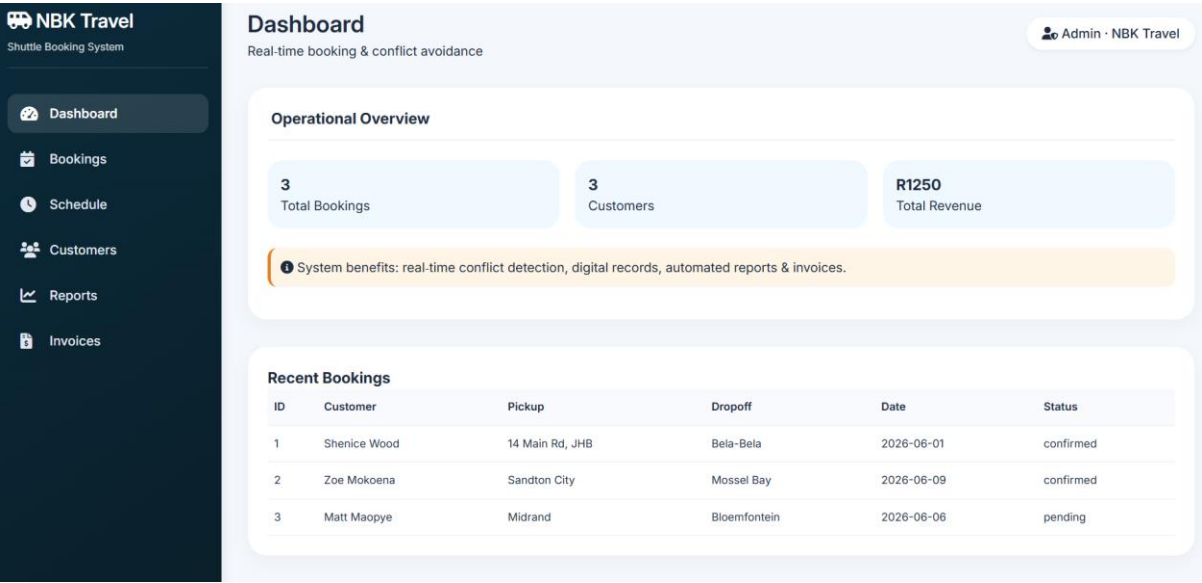
Report Name	Description	Data (MySQL)	Source	Output Format	PHP Method
Trip Count Report	Count of trips per day/week/month	bookings (bookingDate, status)		Screen Chart.js bar/line + print view	generateTripReport(\$start, \$end, \$groupBy)
Revenue Summary Report	Total income per fare period	bookings.fareAmount grouped by date		Screen Chart.js bar chart + print/PDF	generateRevenueReport(\$start, \$end)
Top Customers Report	Ranked list by booking frequency	customers JOIN bookings COUNT(*)		Screen HTML ranked table	getTopCustomers(\$limit)
Driver Utilisation Report	Trips and hours per driver in period	schedules JOIN bookings WHERE driverId		Screen HTML table	getDriverStats(\$driverId, \$start, \$end)
Invoice / Receipt (PDF)	Itemised receipt for completed booking	bookings + customers + invoices		PDF download via html2pdf.js	generateInvoice(\$bookingId)
Notification Audit Log	All simulated notifications sent	notifications table (full log)		Screen HTML read-only table	getNotificationLog(\$start, \$end)

11.1 Report Design Notes

- All screen-based reports use Chart.js (loaded via CDN) for graphical rendering within the HTML/CSS/JS frontend.
- Print views are triggered via the browser's native window.print() dialog, with print-specific CSS media queries hiding navigation and admin controls.
- PDF invoices are generated client-side using html2pdf.js (loaded via CDN), converting the rendered HTML invoice template into a downloadable PDF. This approach was selected for its zero-cost, zero-server-dependency architecture (no additional PHP libraries required).
- All report queries are implemented as PHP functions with MySQLi prepared statements, preventing SQL injection and ensuring data integrity.
- Report access is restricted to the Administrator role via PHP session authentication. Drivers access only their own trip data through the driver dashboard.

12. Prototype

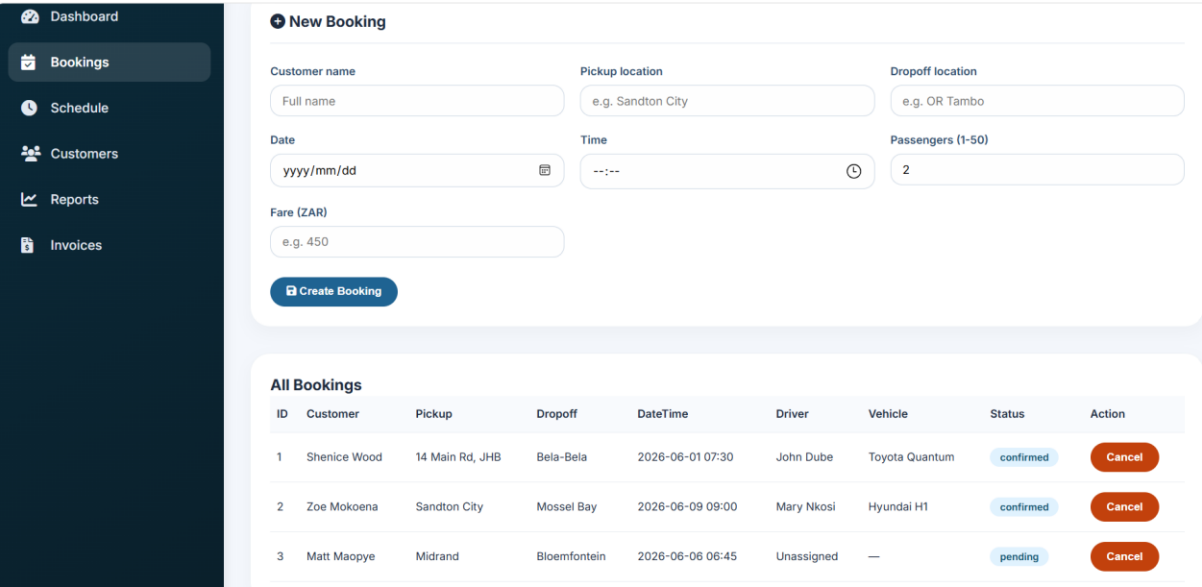
12.1 Dashboard



Central command center conveying:

- **Key metrics:** Total bookings, active customers, revenue summary, upcoming trips
- **Recent activity:** Latest 5 bookings with status (pending/confirmed/cancelled)
- **Quick-access buttons** to all major features

12.2 Bookings



Complete booking management:

- **Create new booking** - Customer details, pickup/dropoff, date/time, passengers, fare
- **View all bookings** - Sortable table with all reservations
- **Cancel bookings** - One-click cancellation with reason logging
- **Real-time status** - Pending → Confirmed → Completed → Cancelled

12.3 Schedule

NBK Travel
Shuttle Booking System

Schedule & Assignment
Real-time booking & conflict avoidance

Admin · NBK Travel

Assign Driver & Vehicle

Select Booking ID: #3 - Matt Maopye (2026-06-06) | Driver: John Dube (available) | Vehicle: Toyota Quantum (16 seater)

Assign & Detect Conflict

✓ No conflict. Assignment successful.

Weekly Schedule Grid

Booking ID	Pickup → Dropoff	Date/Time	Driver	Vehicle	Conflict
2	Sandton City → Mossel Bay	2026-06-09 09:00	Mary Nkosi	H1	✓ clear
1	14 Main Rd, JHB → Bela-Bela	2026-06-01 07:30	John Dube	Quantum	✓ clear

Driver & vehicle assignment with conflict prevention:

- **Assign drivers/vehicles** to confirmed bookings
- **Conflict detection** - Alerts if driver or vehicle already booked at same time
- **Weekly schedule grid** - Visual overview of all assigned trips
- **Availability tracking** - Shows which drivers/vehicles are free

12.4 Customers

The screenshot shows the 'Customer Records' page in the NBK Travel Shuttle Booking System. The left sidebar contains navigation links: Dashboard, Bookings, Schedule, Customers (highlighted), Reports, and Invoices. The main content area is titled 'Customer Records' with the subtitle 'Real-time booking & conflict avoidance'. In the top right corner, it says 'Admin · NBK Travel'. Below the title is a 'Customer Directory' section with a search bar labeled 'Search by name or phone...'. A 'Search' button is located below the search bar. The directory contains a table with the following data:

ID	Full Name	Phone	Email	Bookings Count
1	Shenice Wood	0608178335	jessicawood07@icloud.com	1
2	Zoe Mokoena	0712345678	ZM@nbktravel.co.za	1
3	Matt Maopye	0834567890	matt.n@work.co.za	1

Centralized client database:

- **Customer directory** - Search by name or phone number
- **Booking history** - View all past trips per customer
- **Add/Edit profiles** - Contact details and preferences
- **Repeat customer recognition** - Auto-suggest existing customers

12.5 Reports

The screenshot shows the 'Trip & Revenue Reports' page in the NBK Travel Shuttle Booking System. The left sidebar contains navigation links: Dashboard, Bookings, Schedule, Customers, Reports (highlighted), and Invoices. The main content area is titled 'Trip & Revenue Reports'. It features two input fields for 'Start Date' and 'End Date', both with a calendar icon. Below these fields is a button labeled 'Generate Revenue & Trip Charts'. Below the button is a table titled 'Top Customers by Booking Frequency' with the following data:

Customer	Total Bookings	Total Spent
Shenice Wood	1	R180.00
Zoe Mokoena	1	R450.00
Matt Maopye	1	R620.00

Business intelligence & analytics:

- **Revenue report** - Total income by date range (chart view)
- **Trip volume report** - Number of trips per day/week/month
- **Top customers** - Ranked by booking frequency and total spend
- **Driver utilization** - Trip counts per driver

12.6 Invoices

NBK Travel
Shuttle Booking System

Invoices & Notifications
Real-time booking & conflict avoidance

Admin · NBK Travel

Generate Invoice (PDF)

Select Booking ID for Invoice: 1 - Shenice Wood (R180) [Download PDF Invoice](#)

NBK Travel Invoice

Invoice #INV-1
Date: 2026/05/12
Customer: Shenice Wood (0608178335)
Trip: 14 Main Rd, JHB → Bela-Bela
Date/Time: 2026-06-01 07:30
Passengers: 3
Total Fare: R180.00
Status: confirmed
Thank you for choosing NBK Travel.

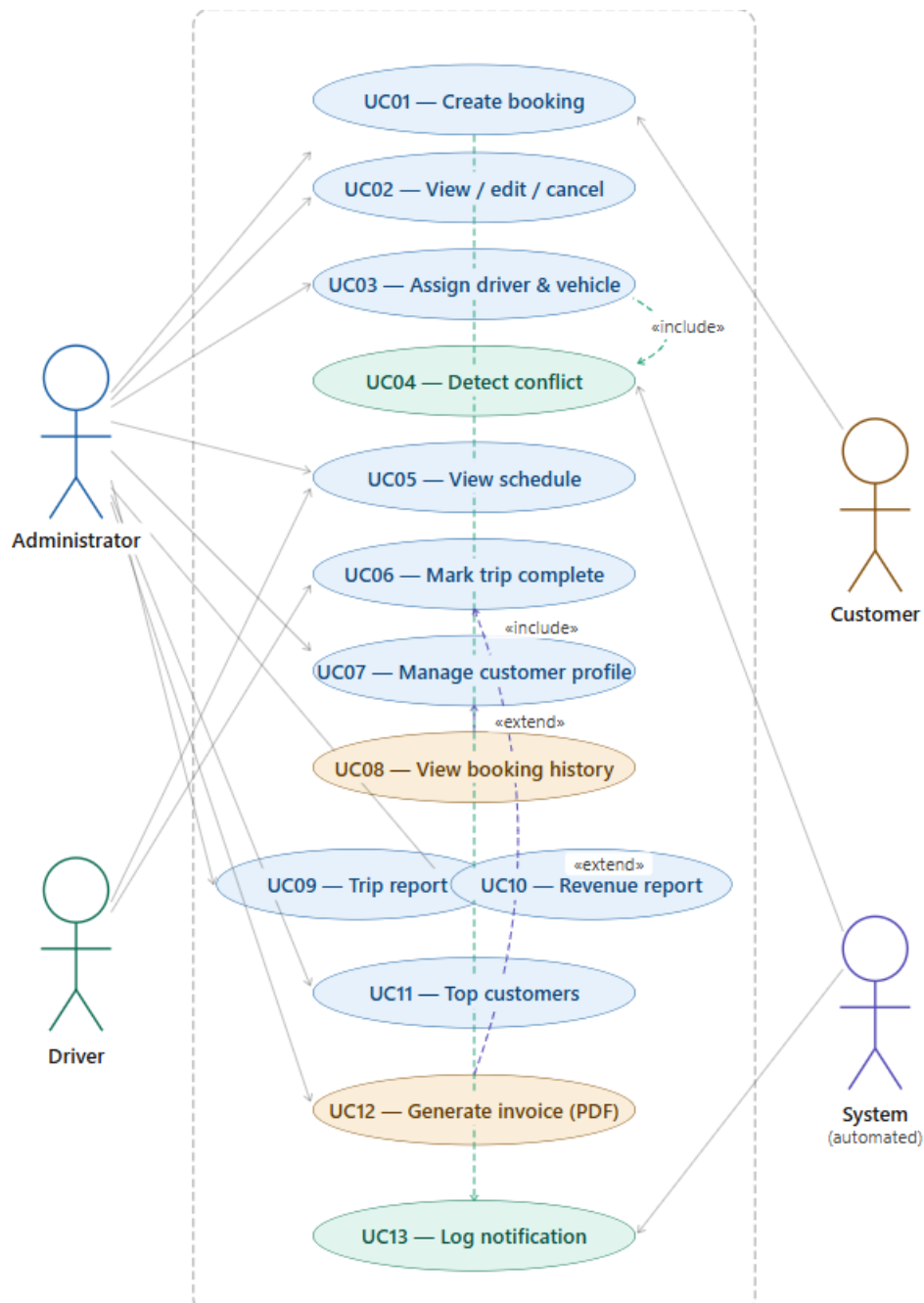
Automated system:

- **PDF generation** - Professional invoices via html2pdf.js
- **Auto-populated** - Customer details, trip info, fare amount
- **Download/print** - Save or print for customer records
- **Email simulation** - Logged notification when invoice generated

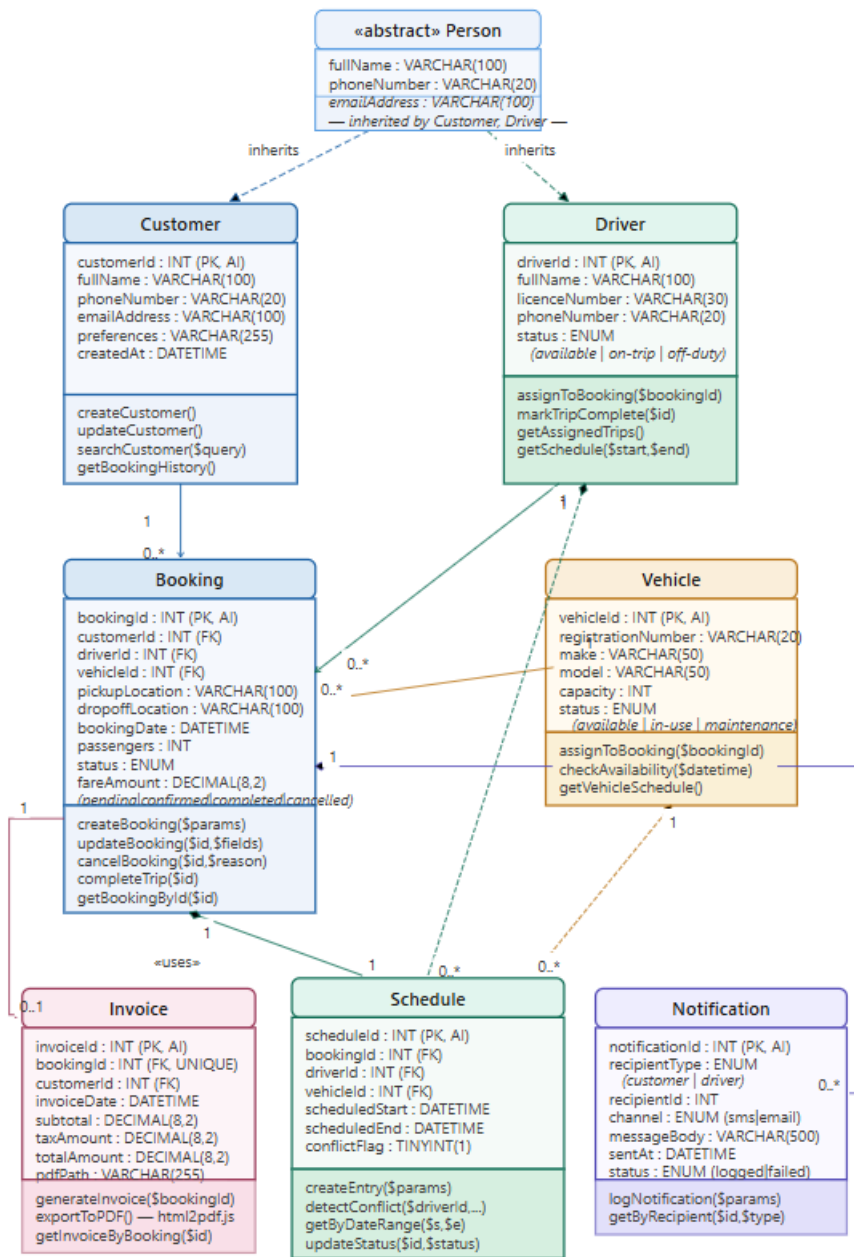
13. Appendix

- Appendix A
- Appendix B
- Appendix B
- References

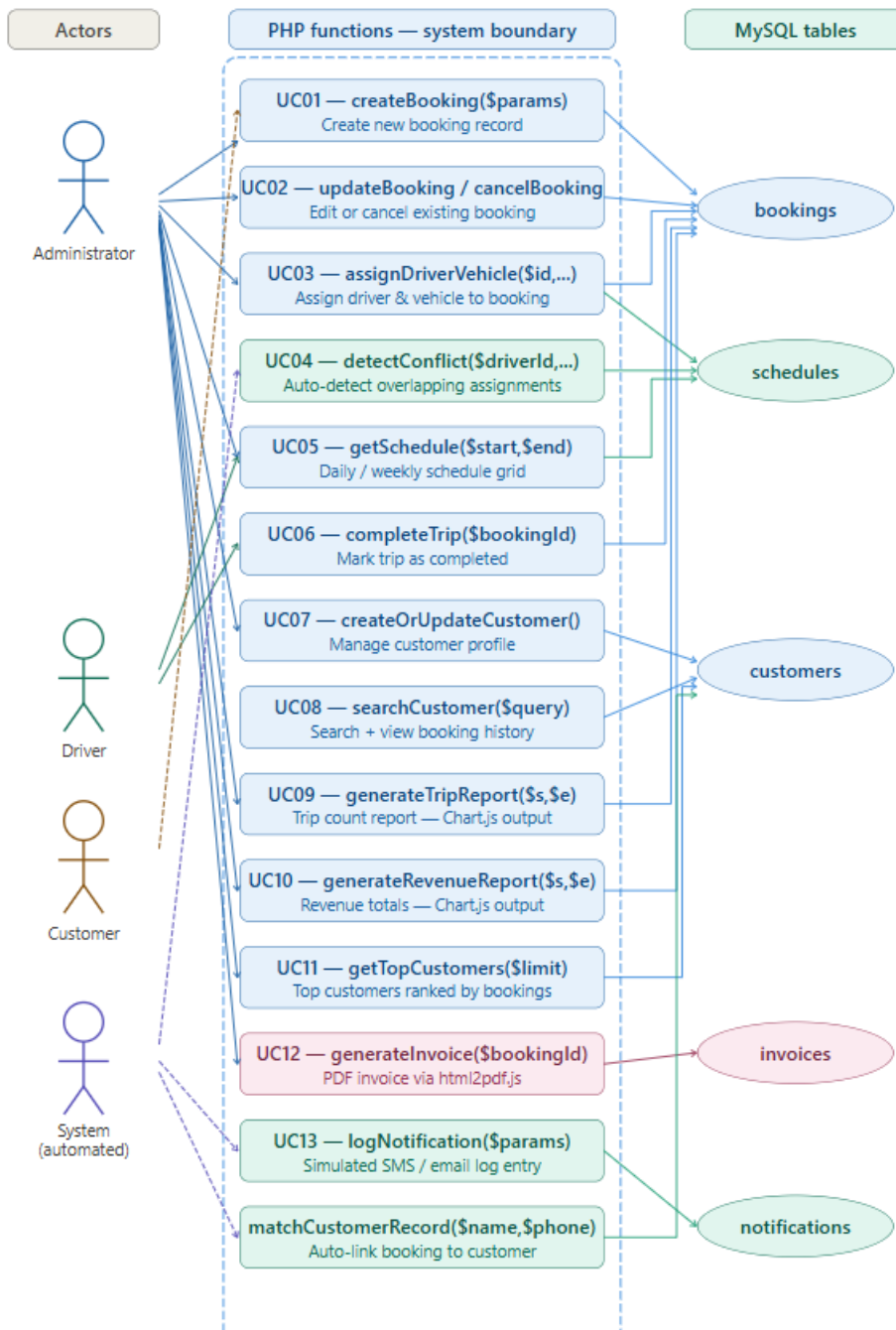
Appendix A: UML Use Case



Appendix B: Class Diagram



Appendix C: Low-Level Design Diagram



References

draw.io (2024). draw.io - Diagram Software and Flowchart Maker. [Online] Available at: [https:// draw.io](https://draw.io) [Accessed 11 April 2026].

Project Management Institute (2021). *A Guide to the Project Management Body of Knowledge (PMBOK Guide)*, 7th Edition. PMI.

The Independent Institute of Education (Pty) Ltd (2026). *WIL Module Manual XISD5319w*. Johannesburg: IIE.